Reto procesamiento

Parque Eólico

Proyecto IoT — Python · FastAPI · SQLAlchemy · Streamlit · Pydantic

FastAPI | Pydantic v2 | SQLAlchemy | Streamlit | Python 3

Arquitectura del Sistema

Tres capas independientes · HTTP REST · SQLite

SENSORES

⚡

`generator.py`

Python + threading
10 hilos paralelos
PROB_ERROR = 0.10

→ *POST /ingesta X-Api-Key*

CONCENTRADOR

🔀

`main.py`

FastAPI + Pydantic
SQLAlchemy + SQLite
API Key auth

→ *GET /estadisticas X-Api-Key*

DASHBOARD

📊

`dashboard.py`

Streamlit
polling 3s
session_state

Modelo de Datos — Pydantic v2

Validación declarativa · Rechazo automático de datos inválidos

```python
# models.py — Validación con Pydantic v2
from pydantic import BaseModel, Field
from typing import Literal

class Lectura(BaseModel):
    id_generador: int = Field( ... , ge=1, le=10)
    timestamp: datetime
    potencia_kw: float = Field( ... , ge=0.0)
    estado: Literal[
        "operativo",
        "mantenimiento",
        "avería"
    ]

    @field_validator("timestamp")
    def check_tz(cls, v):
        if v.tzinfo is None:
            raise ValueError("Requiere zona horaria")
        return v
```

🔢 id_generador: 1–10

Field(ge=1, le=10)
Rechaza 0, 11, 99...

⚡ potencia_kw ≥ 0.0

Field(ge=0.0)
No existe potencia negativa

🏷️ estado controlado

Literal[3 valores]
Rechaza cualquier otro string

🕐 timestamp con TZ

@field_validator
Lanza ValueError si tzinfo=None

El Concentrador — FastAPI

Ingesta · Validación · Persistencia · Seguridad · Agregación SQL

```python
# Seguridad: API Key como Dependency
def verificar_api_key(
    x_api_key: str = Header( ... )
):
    if x_api_key ≠ API_KEY:
        raise HTTPException(status_code=401)

@app.post("/ingesta", status_code=201)
def ingestar_lectura(
    lectura: Lectura,  # Pydantic valida aquí
    db: Session = Depends(get_db),
):
    registro = LecturaDB(**lectura.dict())
    db.add(registro)
    db.commit()
    return {"ok": True}
```

```python
@app.get("/estadisticas")
def estadisticas(db = Depends(get_db)):
    hace_1min = datetime.utcnow()
                - timedelta(minutes=1)

    return db.query(
        func.avg(LecturaDB.potencia_kw),
        func.sum(LecturaDB.potencia_kw),
        func.count(LecturaDB.id),
    ).filter(
        LecturaDB.timestamp ≥ hace_1min
    ).one()
```

`422`	Rechazo automático Pydantic
`Depends()`	Inyección de dependencias
`func.avg`	Agregación SQL ventana 1min

Los Generadores — Datos Sintéticos

10 aerogeneradores · threading · PROB_ERROR = 0.10

```python
PROB_ERROR = 0.1

def generar_valida(id_gen):
    # Distribución gaussiana N(1200, 400)
    kw = max(0, min(2000, random.gauss(1200, 400)))

def bucle(id_gen):
    while True:
        es_error = random.random() < PROB_ERROR
        payload = (
            generar_erronea(id_gen)
            if es_error else
            generar_valida(id_gen)
        )
        requests.post(URL, json=payload)
        time.sleep(2)

# Lanzar 10 generadores en hilos
for i in range(1, 11):
    Thread(target=bucle, args=(i,), daemon=True).start()
```

Tipos de error inyectados

potencia_string

Campo: potencia_kw → valor: "MUCHO_VIENTO"

id_fuera_rango

Campo: id_generador → valor: 0, 11, 99

estado_invalido

Campo: estado → valor: "girando_rapido"

timestamp_sin_tz

Campo: timestamp → valor: datetime.now()

FastAPI responde 422 Unprocessable Entity a cada error

Persistencia — SQLAlchemy + SQLite

ORM · Modelo relacional · Sesión por petición con Depends

```python
# Definición del modelo ORM
class LecturaDB(Base):
    __tablename__ = "lecturas"

    id          = Column(Integer, primary_key=True)
    id_generador = Column(Integer, nullable=False)
    timestamp   = Column(DateTime, nullable=False)
    potencia_kw = Column(Float, nullable=False)
    estado      = Column(String, nullable=False)

# Sesión por petición (sin leaks)
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()
```

Tabla SQLite: lecturas

id	INTEGER PK
id_generador	INTEGER
timestamp	DATETIME
potencia_kw	FLOAT
estado	VARCHAR

⚠️ SQLite no guarda zona horaria.
Solución: convertir a UTC naive antes de persistir.

Dashboard — Streamlit en tiempo real

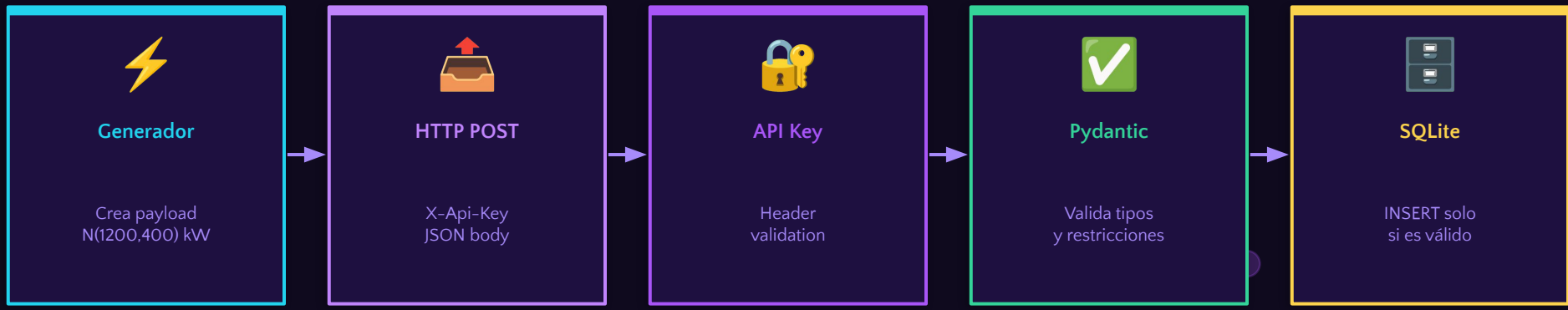
Polling cada 3s · session_state · Gráfico · Semáforo

```python
# Historial en memoria entre refresos
if "historial" not in st.session_state:
    st.session_state.historial = deque(maxlen=30)

while True:  # Bucle de refresco
    datos = requests.get(
        "http://localhost:8000/estadisticas",
        headers=HEADERS
    ).json()

    media = datos["parque"]["media_potencia_kw"]
    st.session_state.historial.append(media)

    st.metric("Media parque", f"{media:.1f} kW")
    st.line_chart(historial_df)

    time.sleep(3)
    st.rerun()
```

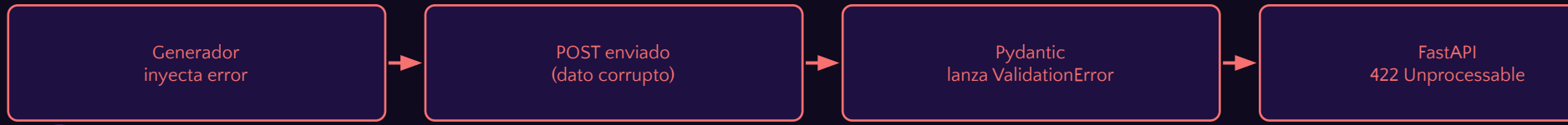
Generador	Media kW	Lecturas
Turbina 01	406.04	12
Turbina 02	1060.05	13
Turbina 03	781.26	12
Turbina 04	752.17	13
Turbina 05	895.9	9
Turbina 06	757.71	13
Turbina 07	874.19	12
Turbina 08	754.44	12
Turbina 09	714.19	13
Turbina 10	900.82	13

Flujo Completo — Dato válido vs. inválido

Ciclo de vida de una lectura a través del sistema



Camino del dato INVÁLIDO (PROB_ERROR = 10%)



❌ *El dato NO se escribe en la base de datos*

Instrucciones de Ejecución

Orden correcto de arranque de los tres componentes

01

Instalar dependencias

```
pip install fastapi uvicorn sqlalchemy
        requests streamlit pydantic pandas
```

02

Arrancar el Concentrador (Terminal 1)

```
python -m uvicorn main:app --reload --port 8000
# Verificar: curl http://localhost:8000/health
```

03

Arrancar los 10 Generadores (Terminal 2)

```
python generator.py
# O un generador concreto:
python generator.py --id 3
```

04

Abrir el Dashboard (Terminal 3)

```
python -m streamlit run dashboard.py
# Abre http://localhost:8501
```

💡 En Windows: usa 'python -m streamlit run' y 'python -m uvicorn' si el PATH no incluye Scripts/

Retos encotrados

SQLite sin zona horaria

SQLite no almacena tzinfo en columnas DateTime

-> Convertir a UTC antes de guardar los datos

Path de ejecutables en windows

streamlit y uvicorn instalados con pip no siempre quedan en el PATH.

->usar python -m streamlit run y python -m uvicorn en lugar de los ejecutables directamente.

💡 Vías de Mejora

Corto plazo

- 🔑 JWT en lugar de API Key estática
- 🧪 Tests unitarios con pytest + httpx
- 🚨 Alertas en tiempo real por avería

Medio plazo

- 📡 MQTT como transporte IoT nativo
- 🗄️ TimescaleDB para series temporales
- ⏱️ Ventanas de agregación variables

Largo plazo

- 🐳 Docker Compose multi-servicio
- 🤖 ML para mantenimiento predictivo
- 🔌 WebSocket → eliminar polling